Constraint Satisfaction Problems Study Notes

Part 2: Modeling CSPs Through Examples

Learning Goal

By the end of this part, you should be able to take a real problem, identify its variables, domains, and constraints, and express it clearly as a Constraint Satisfaction Problem. You should also be able to distinguish between different types of constraints and understand why good modeling is essential for good solving.

1. Core Concept

Modeling is the process of converting a real or abstract problem into the formal structure of a CSP. This means deciding:

- what the variables are,
- what values each variable may take,
- what restrictions must hold among those values.

If the modeling is vague, the solving will also be weak. If the modeling is precise, the problem becomes much easier to analyze and solve.

> **Things to Note:**
>
> - CSP solving starts long before any algorithm is applied. It starts with correct modeling.
> - A badly chosen variable or domain can make an easy problem look hard.
> - The first question is not “Which algorithm should I use?” The first question is “How should I represent the problem?”

2. A General Method for Modeling a CSP

When you are given a problem and asked to represent it as a CSP, a useful method is:

1. Identify the unknown decisions that must be made. These become the **variables**.
2. List the possible values each decision can take. These become the **domains**.
3. Write the rules that limit legal assignments. These become the **constraints**.
4. Check whether any constraints are absolute rules or only preferences.
5. Check whether the problem is asking for any valid solution or the best solution.

> **Modeling Shortcut:**

> If you can answer the three questions “What must be decided?”, “What choices are allowed?”, and “What combinations are forbidden?”, you are already close to a CSP model.

3. Example 1: Map Coloring

In the map-coloring problem, the task is to assign colors to regions so that neighboring regions do not share the same color.

Step 1: Variables

Each region becomes a variable. For example:

- `WA`
- `NT`
- `Q`
- `NSW`
- `V`
- `SA`
- `T`

Step 2: Domains

Each variable can take one of the available colors:

```
{red, green, blue}
```

Step 3: Constraints

Any two neighboring regions must have different colors.

Examples:

- `WA ≠ NT`
- `WA ≠ SA`
- `NT ≠ Q`
- `SA ≠ NSW`

What this model shows

This is a clean example of a **binary CSP** because most constraints involve pairs of variables.

Map Coloring Component	Meaning
Variables	Regions on the map
Domains	Available colors

Constraints	Adjacent regions must have different colors

4. Example 2: Exam Scheduling

Now consider a university exam scheduling problem.

The university must assign time slots to exams in a way that avoids conflicts.

Step 1: Variables

Each course exam becomes a variable:

- `CS101`
- `CS102`
- `MATH201`
- `ENG101`

Step 2: Domains

Each exam can be placed in one of the available time slots:

`{Mon-AM, Mon-PM, Tue-AM}`

Step 3: Constraints

Suppose the following hard constraints apply:

- `CS101 ≠ CS102` because some students take both courses
- `CS101 ≠ MATH201` because some students take both courses
- `CS102 ≠ ENG101` because some students take both courses
- `ENG101 ≠ Mon-AM` because the instructor is unavailable then

Optional soft constraints

- `MATH201` is preferably scheduled in the morning
- Avoid giving students two exams on the same day if possible

Exam Scheduling Component	Meaning
Variables	Exams/courses to be scheduled
Domains	Available time slots
Constraints	Conflict rules, availability rules, and possible preferences

Things to Think About:

- Why is `ENG101 ≠ Mon-AM` different from `CS101 ≠ CS102`?

- Which part of the exam scheduling example makes it more realistic than map coloring?
- If we added rooms as well as time slots, how would the domains change?

5. Comparing the Two Examples

These two problems look different on the surface, but their CSP structure is the same.

Feature	Map Coloring	Exam Scheduling
Variables represent	Regions	Exams
Domain values represent	Colors	Time slots
Main binary constraint	Adjacent regions must differ	Conflicting exams must differ
Unary constraint example	Region cannot use a certain color	Exam cannot use a certain slot
Soft constraint example	Prefer one color over another	Prefer a morning slot

The lesson is simple: different real-world problems can often be mapped onto the same formal CSP framework.

6. Constraint Graphs

A very useful way to understand a CSP is through its **constraint graph**.

In a constraint graph:

- nodes represent variables,
- edges represent constraints between variables.

For map coloring, each node is a region and each edge shows adjacency.

For exam scheduling, each node is an exam and each edge shows that two exams cannot be scheduled in the same slot.

Why constraint graphs matter

- They help visualize which variables interact.
- They reveal independent subproblems.
- They make structural reasoning possible.

Things to Note:

- If two variables are not connected by a constraint, they may often be treated more independently.

- Constraint graphs become especially important later when studying decomposition and tree-structured CSPs.

7. Types of Constraints in More Detail

Unary Constraints

These involve only one variable.

Example: `ENG101 ≠ Mon-AM`

This removes a value directly from one variable's domain.

Binary Constraints

These involve two variables.

Example: `CS101 ≠ CS102`

This is the most common kind of constraint in introductory CSP problems.

Higher-Order Constraints

These involve three or more variables at the same time.

Example: "At most two exams may be scheduled in the same time slot."

Such constraints are more complex because they cannot always be expressed as simple pairwise restrictions.

Soft Constraints

These represent preferences rather than strict rules.

Example: "Prefer `MATH201` in the morning."

Violating a soft constraint does not make a solution invalid. It just makes it less desirable.

> **Common Confusion:**
>
> Students often think all constraints are hard constraints. That is false. A hard constraint must be satisfied. A soft constraint is only a preference. Breaking a soft constraint may still leave the assignment valid, just not optimal.

8. Finite, Infinite, and Continuous Domains

Not all CSPs have small finite domains.

Finite domains

Examples include colors, digits, rooms, or limited time slots.

These are the easiest to visualize and are common in introductory problems.

Infinite domains

Examples include all integers or all possible dates. Scheduling and planning problems sometimes use such domains.

Continuous domains

Examples include real-valued times, distances, or measurements.

These often appear in engineering or scientific scheduling problems.

The domain type matters because it affects what algorithms are practical and what kinds of constraints can be handled efficiently.

> **Key Idea:**
>
> The same modeling logic applies across different domain types, but the computational difficulty can change significantly depending on whether domains are finite, infinite, or continuous.

9. Good Modeling vs Bad Modeling

A good model makes constraints clear, keeps domains manageable, and matches the real decisions that need to be made.

A bad model may:

- choose the wrong variables,
- make domains unnecessarily large,
- hide useful structure,
- mix hard rules and preferences carelessly.

Example of a modeling mistake

In exam scheduling, if you define one variable as “the entire exam timetable” instead of giving each exam its own variable, you lose the local structure that makes CSP solving efficient.

That would turn a structured problem into an unhelpful black box.

> **Things to Note:**
>
> - Choose variables that represent individual decisions.
> - Keep domains as small as the real problem allows.
> - Separate hard constraints from preferences clearly.
> - Good representation often matters as much as good algorithms.

10. Things to Think About

- If two different real-world problems can be modeled as CSPs, what does that suggest about the usefulness of the CSP framework?
- Why is a unary constraint often easier to handle than a binary one?
- What happens if the domain you choose is much larger than necessary?
- Why does good modeling improve later solving methods such as backtracking or propagation?

11. Quick Recap

- Modeling a CSP means defining variables, domains, and constraints clearly.
- Map coloring and exam scheduling are structurally similar even though they look different.
- Constraint graphs help visualize interactions among variables.
- Constraints may be unary, binary, higher-order, or soft.
- Domain type matters: finite, infinite, and continuous domains may require different treatment.
- Bad modeling can make a solvable problem unnecessarily difficult.

12. Part 2 Revision Lines

> - When modeling a CSP, always ask: **What are the variables? What are their domains? What constraints restrict them?**
> - Map coloring is the clean binary example. Exam scheduling is the practical real-world example.
> - Good modeling exposes structure. Bad modeling hides it.

End of Part 2